

Capitolo 2 — Crescita e algoritmi

Marco Lapegna

[← Torna all'indice](#)

1 La ricerca binaria

La ricerca binaria (o ricerca dicotomica) è un algoritmo che determina la **posizione** pos di un **elemento** X in un **array ordinato** A di **lunghezza** N . Nel caso in cui X non è presente in A , l'algoritmo restituisce convenzionalmente $pos = -1$. La preconditione fondamentale è che l'array sia ordinato: senza di essa l'algoritmo non funziona. L'idea è quella del “divide et impera”. Più precisamente, invece di scorrere l'array elemento per elemento (come nella ricerca sequenziale), l'algoritmo confronta l'elemento cercato con quello che si trova in posizione centrale dell'intervallo di ricerca corrente. Si presentano tre casi. Se l'elemento centrale è uguale a quello cercato, allora si è trovata la posizione e la ricerca termina con successo. Altrimenti, se l'elemento cercato è minore di quello centrale, esso può trovarsi solo nella metà sinistra, quindi la metà destra viene scartata. Se invece è maggiore, può trovarsi solo nella metà destra, e viene scartata la metà sinistra. Il procedimento si ripete sulla metà rimanente, dimezzando a ogni passo l'intervallo di ricerca, finché l'elemento viene trovato oppure l'intervallo di ricerca corrente si svuota, condizione che indica che l'elemento non è presente.

Esempio

Ricerca dell'elemento $X = 23$ in un array A di $N = 8$ elementi.

Algoritmo

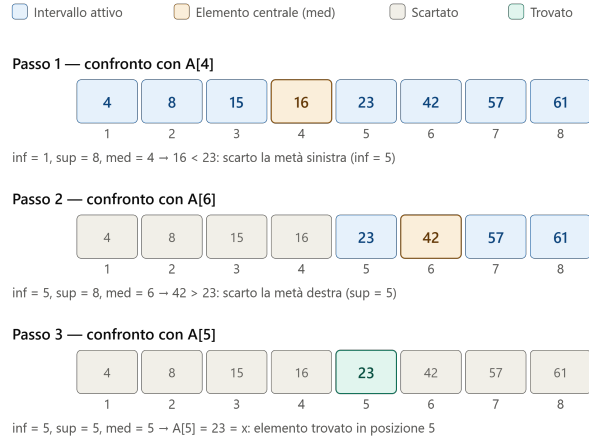


Figura 1: ricerca binaria

Algoritmo 1 Ricerca binaria

```

1: procedure ricbin(in :  $N$ ,  $A$ ,  $X$ ; out : pos)
2: var: inf, sup, med, pos integer
3: var:  $X_{\text{real}}$ 
4: array:  $A[N]$  real
5:
6: inf = 1
7: sup =  $N$ 
8: pos = -1
9: while (inf ≤ sup and pos == -1) do
10:   med =  $\lfloor (\textit{inf} + \textit{sup}) / 2 \rfloor$ 
11:   if ( $A[\textit{med}] == X$ ) then
12:     pos = med
13:   else
14:     if ( $A[\textit{med}] < X$ ) then
15:       inf = med + 1
16:     else
17:       sup = med - 1
18:     end if
19:   end if
20: end while
21: end procedure

```

- le variabili *inf* e *sup* rappresentano gli estremi dell'intervallo di ricerca
- la variabile *med* è l'indice dell'elemento centrale di A
- La struttura *while* (riga 9) determina il criterio di arresto (intervallo di

ricerca vuoto oppure elemento trovato)

- La struttura di selezione a riga 11 confronta X con l'elemento centrale di A
- La struttura di selezione a riga 14 verifica se X è precedente o successivo all'elemento centrale

Complessità computazionale

- **Caso migliore:** l'elemento viene trovato subito in posizione centrale, quindi $T(N) = 1$ confronti, cioè $T(N) \in O(1)$.
- **Caso peggiore:** poiché a ogni iterazione l'intervallo di ricerca si dimezza, la complessità è $T(N) = 2 \log(N)$ cioè $T(N) \in \Omega(\log(N))$ (se N è potenza di 2), contro $T(N) = N$ della ricerca sequenziale.

2 L'ordinamento per selezione

L'ordinamento per selezione ordina un **array** A di **lunghezza** N operando *in place* (cioè senza l'utilizzo di array supplementari). Esso opera per passi dove, ad ogni passo, l'array risulta partizionato in due sezioni. Detto i l'indice del passo, la sezione $A[1, \dots, i-1]$ risulta già ordinata, mentre la rimanente $A[i, \dots, N]$ è ancora da ordinare. L'algoritmo, allora, **seleziona** l'elemento minimo nella sezione non ordinata $A[i..N]$, individuandone la posizione pos , e lo sposta in posizione i mediante uno scambio con $A[i]$. Dopo lo scambio la porzione ordinata dell'array si allunga di una posizione e contiene gli i elementi più piccoli dell'intero array, già nella loro posizione ordinata definitiva, mentre la sezione non ordinata si riduce di un elemento. Dopo $N - 1$ iterazioni l'elemento in posizione N è necessariamente il massimo e l'array risulta ordinato.

Esempio

Ordinamento di un array A di $N = 5$ elementi.

Algoritmo



Figura 2: ordinamento per selezione

Algoritmo 2 Ordinamento per selezione

```

1: procedure ordsel(in :  $N, A$ ; out :  $A$ )
2: var:  $i, j, pos$  integer
3: var:  $T$  real
4: array:  $A[N]$  real
5:
6: for  $i = 1$  to  $N - 1$  do
7:    $pos = i$ 
8:   for  $j = i + 1$  to  $N$  do
9:     if  $A[j] < A[pos]$  then
10:       $pos = j$ 
11:    end if
12:  end for
13:  if  $pos \neq i$  then
14:     $T = A[pos]$ 
15:     $A[pos] = A[i]$ 
16:     $A[i] = T$ 
17:  end if
18: end for
19: end procedure

```

- le istruzioni da 7 a 12 rappresentano la ricerca del minimo nella sezione on ordinata $A[i, \dots, N]$
- le istruzioni da 14 a 16 rappresentano lo scambio tra il minimo in $A[i, \dots, N]$ e $A[i]$

- lo scambio viene effettuato solo se il minimo non è già nella posizione giusta (controllo di riga 13)

Complessità computazionale

L'algoritmo esegue sempre $N - 1$ iterazioni, per cui effettua sempre $T(N) = \sum_{i=1}^{N-1} (N - i) = \frac{N(N-1)}{2}$ confronti, indipendentemente dalla disposizione iniziale dei dati. La complessità asintotica è dunque $T(N) \in O(N^2)$ in ogni caso. Gli scambi sono al più $N - 1$ nel caso peggiore (ad esempio per l'array $A = [2, 3, 4, 5, 1]$) oppure nessuno se l'array è già ordinato. Poiché l'ordinamento avviene in place la complessità di spazio è $S(N) = N$.

3 L'ordinamento per scambi

L'ordinamento per scambi ordina un **array** A di **lunghezza** N operando *in place* (cioè senza l'utilizzo di array supplementari). Esso opera per passi dove, ad ogni passo, l'array risulta partizionato in due sezioni. Detto i l'indice del passo, inizialmente la sezione $A[N - i + 2, \dots, N]$ risulta già ordinata, mentre la rimanente $A[1, \dots, N - i + 1]$ è ancora da ordinare. L'algoritmo, allora, confronta gli elementi adiacenti due a due nella sezione non ordinata, scambiando quelli non in ordine. Dopo la serie di confronti e scambi nell' i -mo passo, la porzione ordinata dell'array si allunga di una unità e contiene nella sezione $A[N - i + 1, \dots, N]$ gli i elementi più grandi dell'intero array, mentre la sezione non ordinata si riduce di un elemento. Dopo $N - 1$ iterazioni l'array risulta ordinato. E' opportuno osservare, però, che se in una iterazione non avvengono scambi, vuol dire che gli elementi adiacenti sono già in ordine due a due e pertanto l'algoritmo può terminare. Ciò si ottiene con una struttura di iterazione *repeat* che termina quando si sono compiute le $N - 1$ previste oppure se non vengono effettuati scambi.

Esempio

Ordinamento di un array A di $N = 5$ elementi.

Algoritmo



Figura 3: ordinamento per selezione

Algoritmo 3 Ordinamento per scambi

```

1: procedure ordsca(in :  $N$ ,  $A$ ; out :  $A$ )
2: var:  $i, j, S$  integer
3: var:  $T$  real
4: array:  $A[N]$  real
5:
6:  $i = 0$ 
7: repeat
8:    $i = i + 1$ 
9:    $S = 0$ 
10:  for  $j = 1$  to  $N - i$  do
11:    if  $A[j] > A[j + 1]$  then
12:       $T = A[j]$ 
13:       $A[j] = A[j + 1]$ 
14:       $A[j + 1] = T$ 
15:       $S = 1$ 
16:    end if
17:  end for
18: until ( $i = N - 1$  or  $S == 0$ )
19: end procedure

```

- la struttura *repeat* (riga 7) determina le iterazioni da eseguire
- la struttura *for* (riga 10) scorre l'array all'interno del passo i
- la variabile S definisce se sono stati effettuati scambi alla iterazione i ($S == 1$) oppure no ($S == 0$)

Complessità computazionale

- **Caso migliore** (array già ordinato) L'algoritmo scorre e confronta tutte le coppie di elementi adiacenti (righe 10 e 11) e non esegue mai scambi e quindi esegue solo l'iterazione con $i = 1$. Ciò comporta $T(N) = N - 1$ confronti + 0 scambi
- **Caso peggiore** (array in ordine inverso) L'algoritmo trova due elementi adiacenti sempre in ordine inverso, e per ogni confronto esegue uno scambio. Esso esegue quindi sempre $N - 1$ iterazioni, in cui effettua $N - i$ confronti, per un totale di $T(N) = \sum_{i=1}^{N-1} (N - i) = \frac{N(N-1)}{2}$ confronti e altrettanti scambi